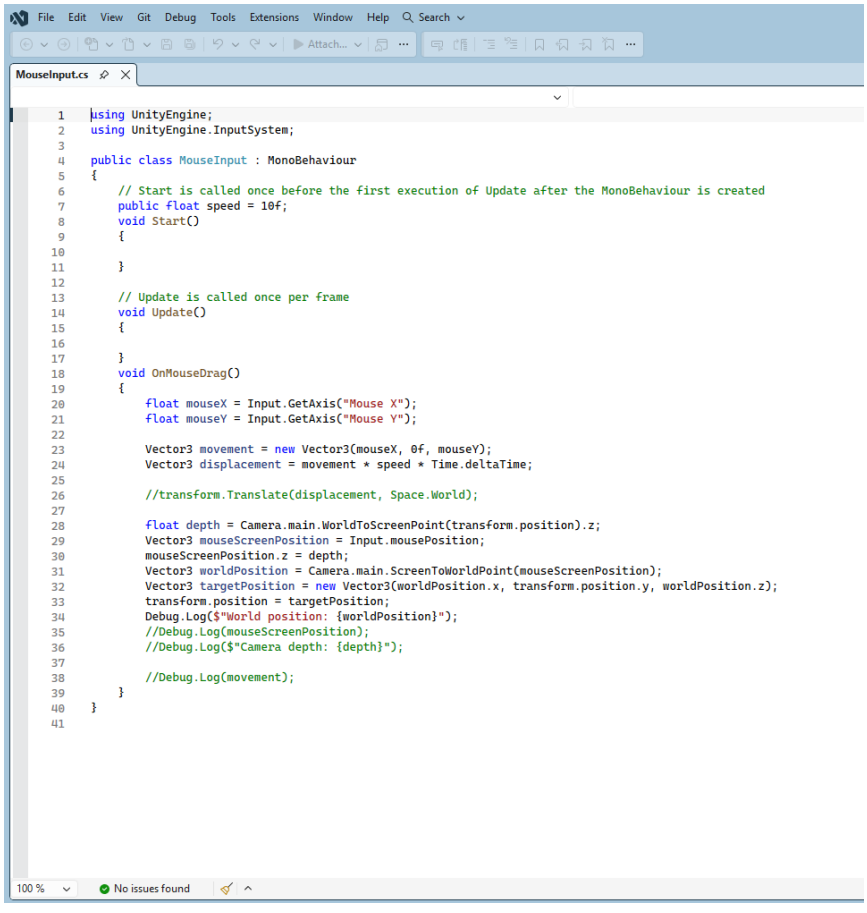
For this coursework, following our first meeting, our team decided to create a LEGO-themed pizza delivery game. In the game, players take on the role of a pizza delivery driver in a city. Their objective is to pick up orders from a pizza takeaway shop and deliver them to designated locations within a set time limit. The final score will be determined by the money earned and customer ratings. Faster deliveries will result in higher earnings, including extra tips and positive customer feedback, while slower deliveries will lead to financial penalties and negative reviews. Both factors will directly influence the final score.

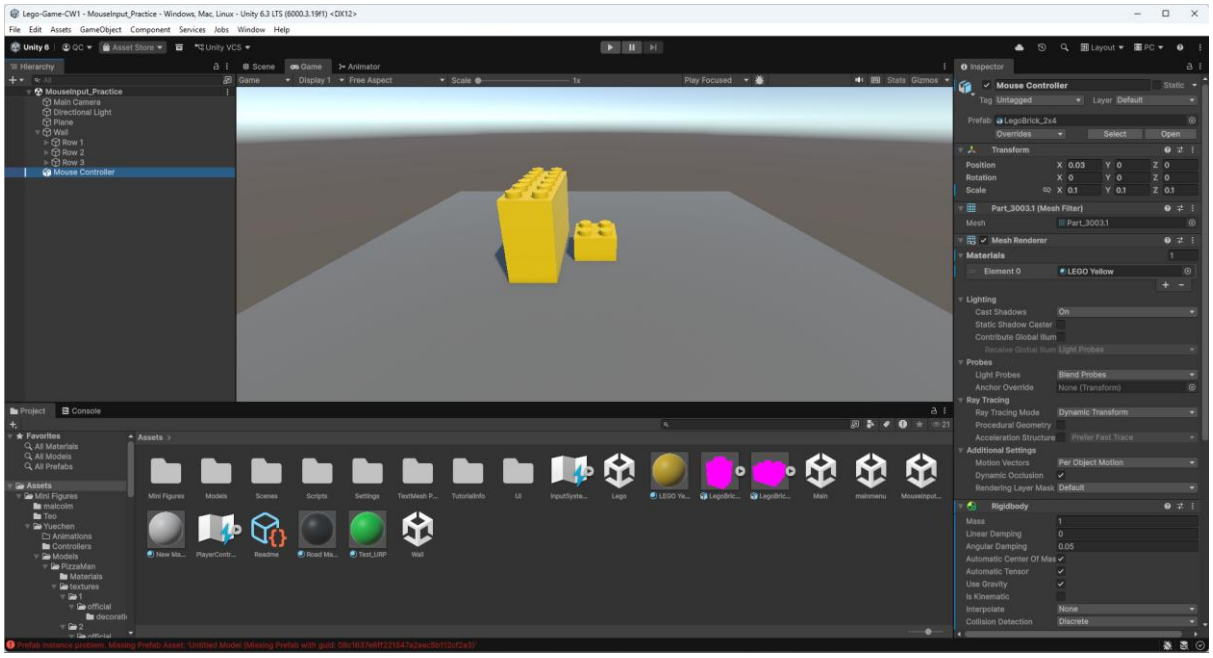
In accordance with the coursework tutorial, I was assigned the task of developing the "Mouse Input Controller", which enables players to drag and move objects using the mouse. I also completed the "Lego Minifigure Animation" task. Additionally, I am responsible for the game's map layout, the construction of buildings and scenes, and the overall environmental design. My goal is to build the foundation of the game map, plan the street layout and placement of various structures, and create original architectural designs. I will also assist other team members with their respective tasks whenever necessary.

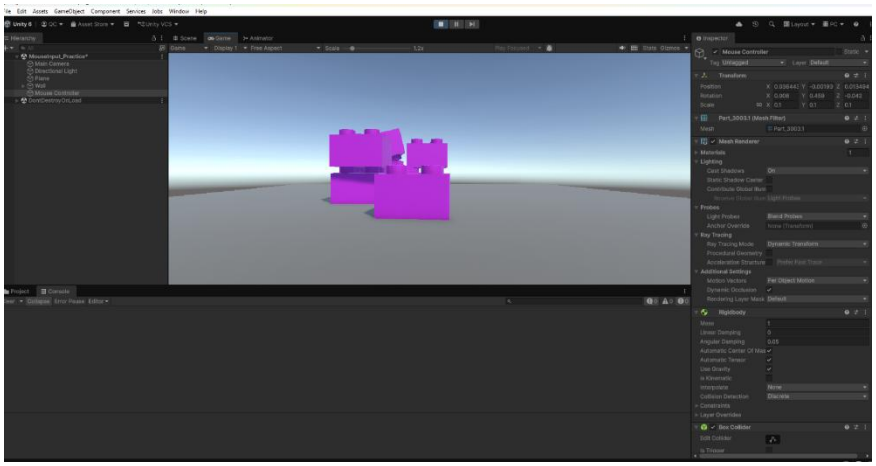
In our game design, mouse input control plays a crucial role. As outlined in the tutorial, this feature allows players to select and drag LEGO bricks, an interaction that could be important throughout the gameplay experience. For instance, our game concept involves picking up a pizza from a pizzeria, placing it onto a motorcycle, and removing it upon reaching the delivery destination to hand it over to the customer. This sequence would require the player to click and move objects, relying heavily on the logic of the Mouse Input Controller.



I began by duplicating an existing object in the scene, renaming it "Mouse Controller", and writing my initial C# script. Through tutorials and further

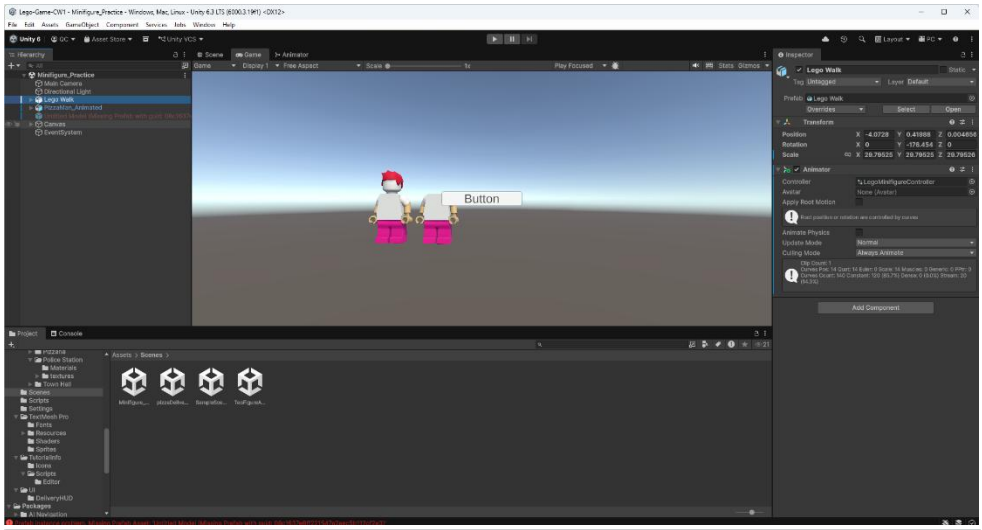
research, I explored two different approaches to object movement: one based on calculating displacement (specifying how far an object should move), and another based on the mouse's screen position and the object's depth relative to the camera (specifying a target position for the object). I initially experimented with the first method but ultimately chose the second. After comparing the two, I concluded that the second approach was better suited to a third-person perspective game, as I found it more intuitive for controlling movement relative to the player's viewpoint.

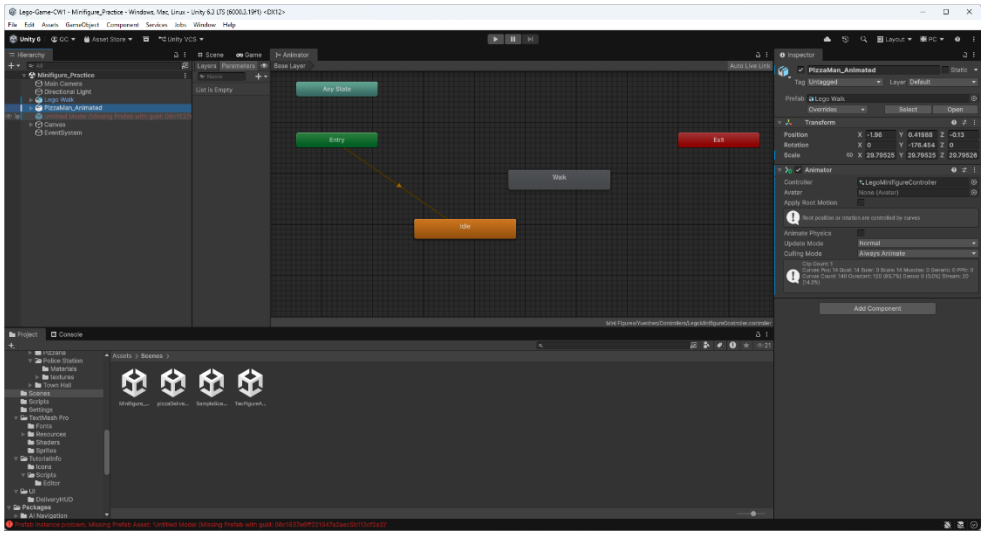


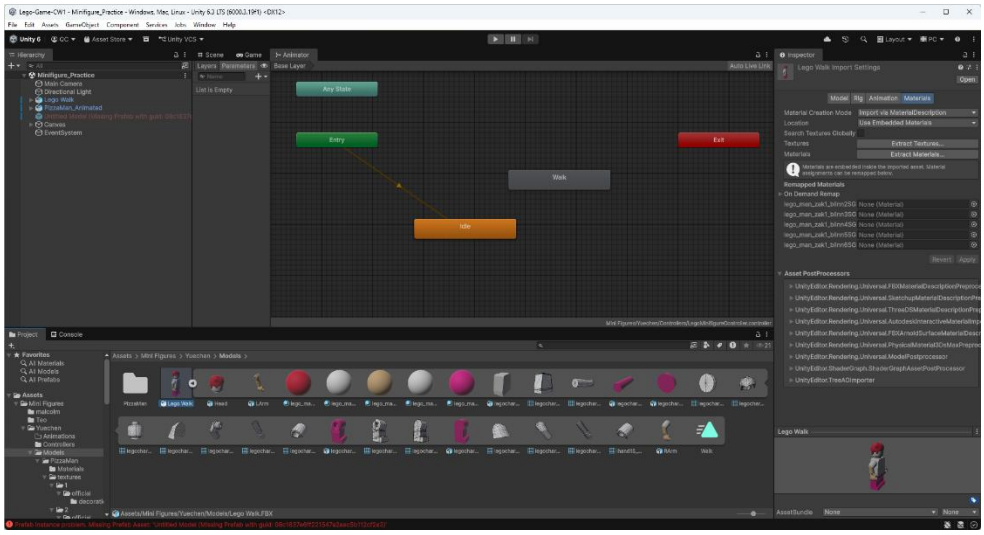


During the coding process, I encountered several challenges. I managed to resolve some of them, while others remain puzzling because I still do not fully understand their root causes. For example, when I first ran the code, the OnMouseDrag() method was not triggered. I discovered that the Unity project was configured to use only the "New Input System", so I changed the setting to "Both" to enable support for both the new and legacy input systems. Another issue involved a wall made of LEGO bricks that was already present in the scene. When I started the game, the wall would unexpectedly break apart, sending bricks flying everywhere. Despite extensive troubleshooting, I could not find a solution. I suspected that overlapping colliders might be responsible, so I decided to temporarily disable the wall to isolate and test the Mouse Input Controller. This allowed me to test

the controller without interference from the physics wall, although I was unable to identify or resolve the underlying cause of the wall breaking apart.

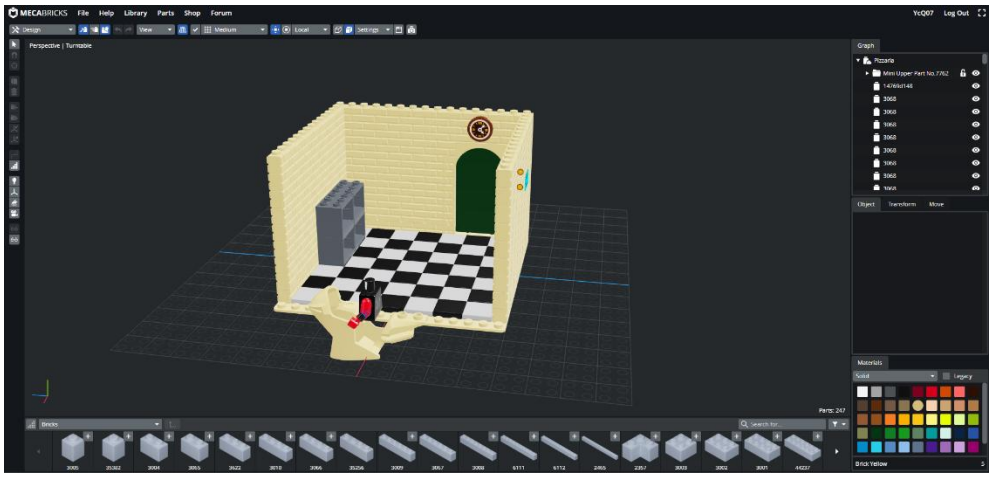


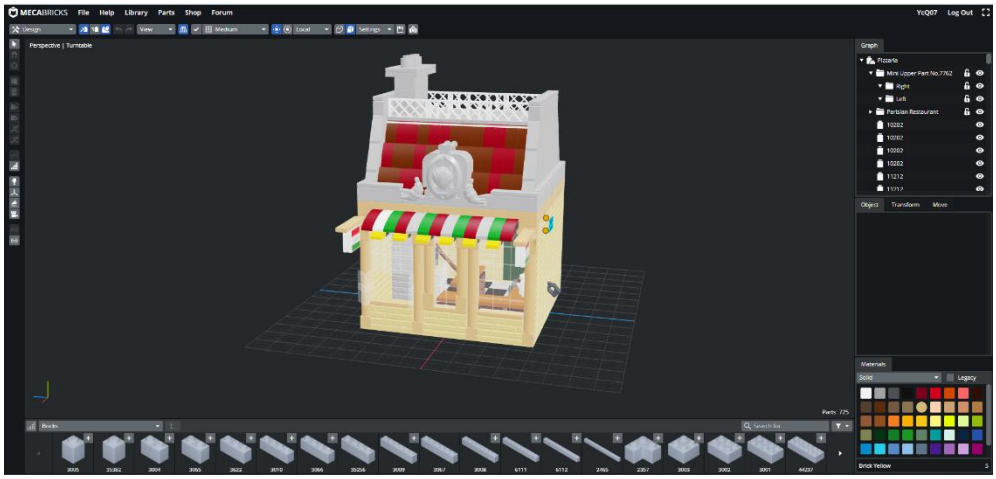




For the Lego Minifigure animation section, I used the "Lego Walk.FBX" file provided in the tutorial. I created a new scene and folder, then imported the file into Unity for testing. Compared with the Mouse Input Controller, learning and implementing the "Lego Walk" animation was much simpler and more straightforward. The tutorial video itself was less than twenty minutes long, and I managed to complete the process with almost no errors. Since the character model and

walking animation were already provided, I mainly needed to import and configure them in Unity, set up the Animator and UI button, and test the results.





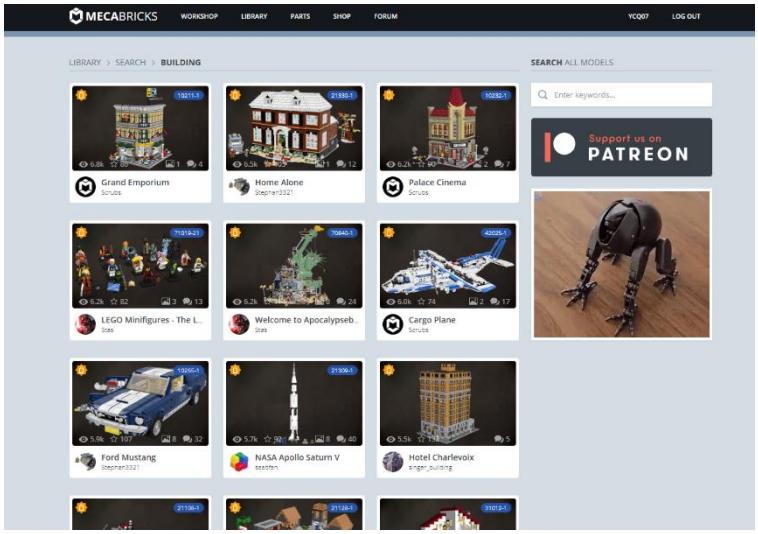
Among my contributions to the group coursework, I am most proud of the original pizza shop I built. It serves as one of the most important structures in the game: the player's starting and ending point, the hub for accepting and completing missions, and a building that establishes the overall atmosphere of the game. In many ways, it acts as the game's signature landmark. Furthermore, this was my first experience of designing and constructing a building entirely from scratch. I had very little prior experience in this area, having spent little time with physical LEGO sets or even *Minecraft*. Therefore, this was a complete design and construction process that I carried out independently, without using any blueprints.



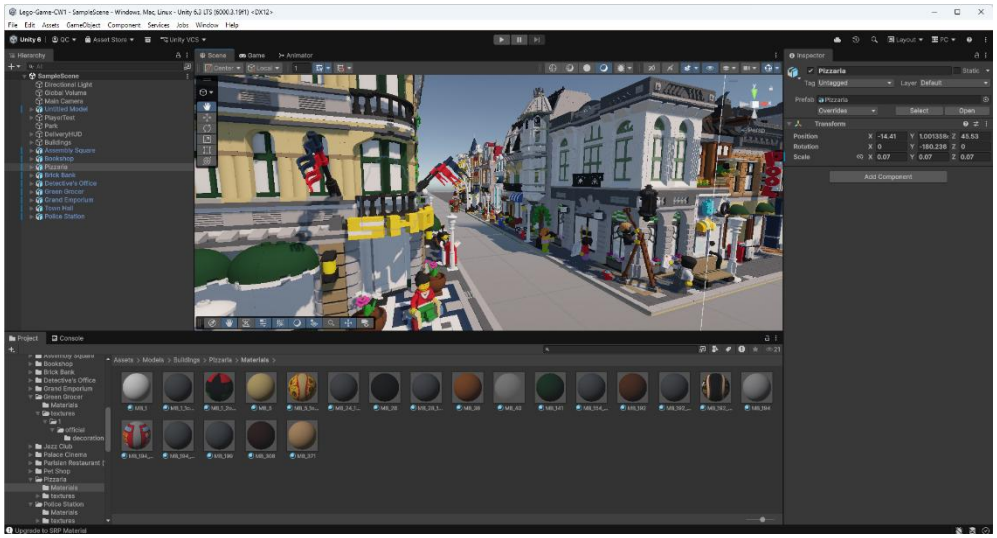
For the style of the pizza shop, I drew inspiration from Italian architecture, incorporating features such as beige brick walls, red roof tiles, white marble-style roof decorations, arched

glass windows with cylindrical frames, and cabinetry in shades of brown and honey. My goal was to make the building stand out on the street while still blending naturally into its surroundings. In terms of scale, I compared my design with other structures available in Mecabricks to determine suitable floor heights and room dimensions. Since the building is a pizza takeaway rather than a dine-in restaurant, and space is limited, I designed it without customer seating or interior access. The entire ground floor functions as a kitchen, where players and other in-game customers can collect pizzas and make payments through an open glass service window.

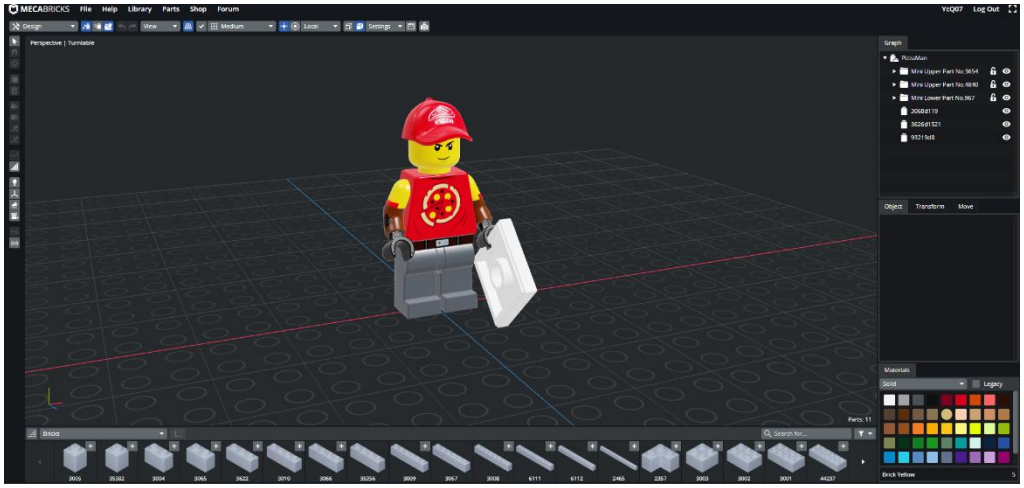
During the construction and decoration process, I researched various LEGO part numbers to find pieces that could achieve the appearance I wanted or serve as suitable alternatives to pieces I could not find. For instance, I wanted to include a pizza peel, but since I could not find a suitable LEGO piece, I followed a workaround commonly used by the community: I took a LEGO oar, recoloured it brown, and placed it next to the oven. I was quite satisfied with the result. I also experimented with other creative techniques, such as repurposing a cupboard as an oven, combining cylindrical pieces to create jars and containers, and even allowing two otherwise incompatible pieces to intersect to achieve the desired appearance. Thanks to the flexible design system, which allowed me to make adjustments at any time, I successfully completed my first architectural build.



Beyond the pizza shop, the map required various other buildings to fill out the city block. I explored the Mecabricks library and found models from the LEGO Modular Buildings series. I

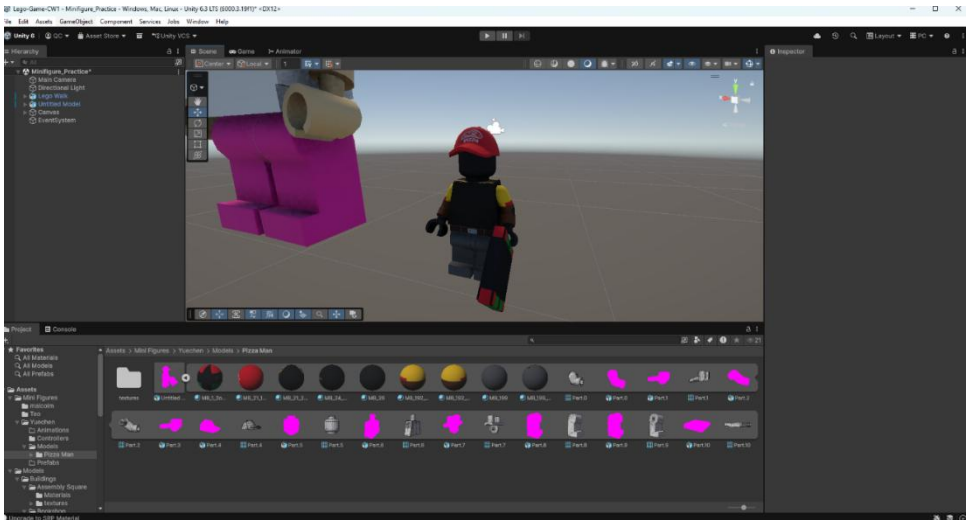


selected buildings that matched our game's requirements in terms of both style and function, then downloaded and imported them into Unity to construct the streetscape. Their architectural styles complemented my pizza shop, helping my original design and the imported buildings come together to create a visually cohesive city environment.



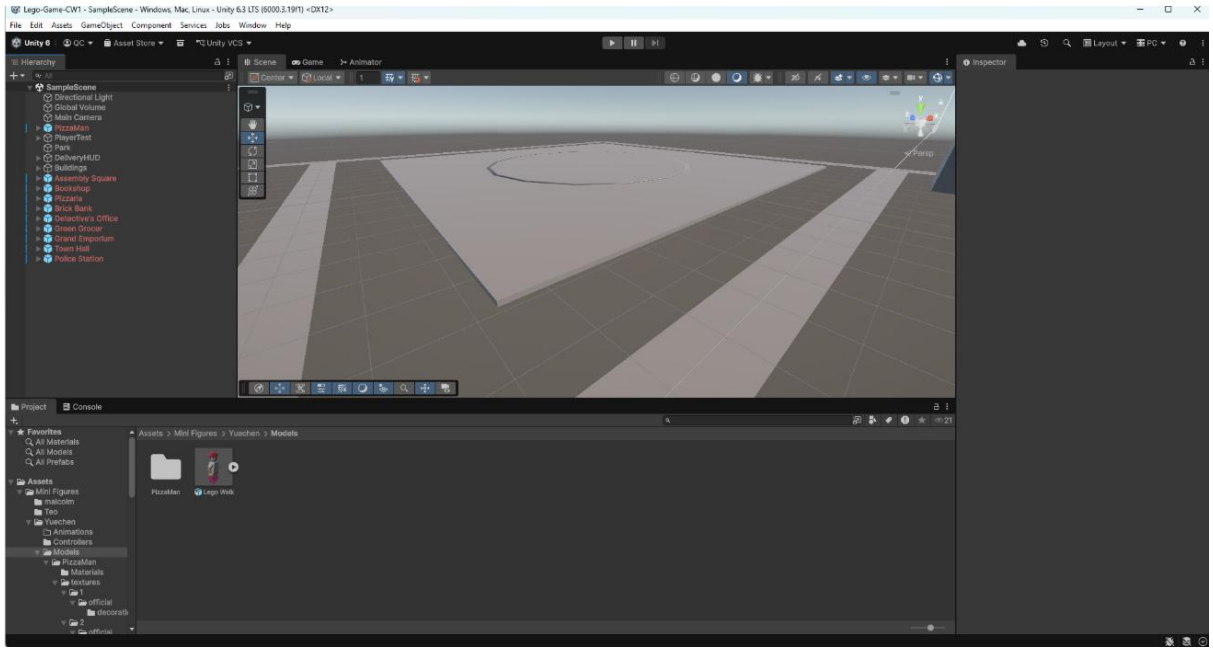
In addition, I created a "pizza delivery person" character that I hope to use as the game's protagonist in the future. For the character's appearance, I used the pizza-themed hat and uniform options available on Mecabricks, while carefully selecting the remaining elements to represent a pizza delivery worker. The design uses a gold and red colour scheme, making the character visually distinctive and easily recognisable. My next goal is to learn how to create animations so that I can give the character a wider range of expressions and movements, which are important for a game protagonist. Although the character model is complete, I have not yet started animating it, as the process has proven more complex than I initially expected. I aim to complete this part of the project before the Coursework 2 deadline.

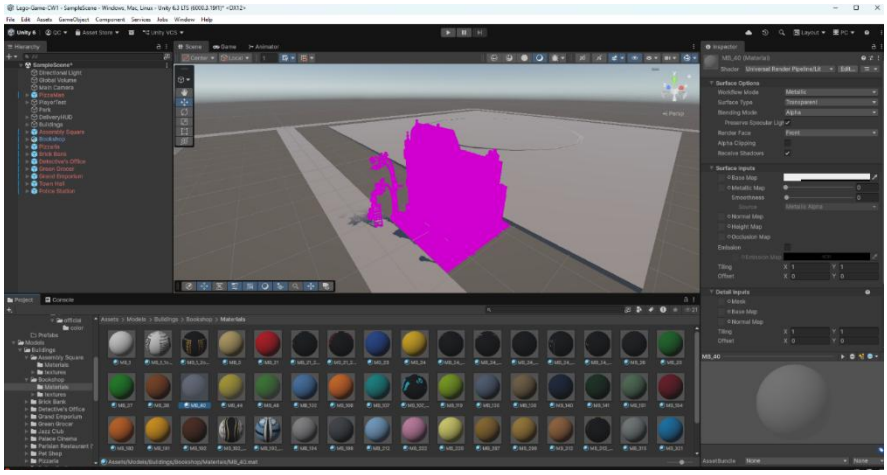




Although building the map and setting up characters might seem relatively straightforward, I still encountered several technical challenges. The most troublesome issue occurred when importing architectural assets: textures that were supposed to contain transparent areas appeared solid black in Unity. For example, parts of a pizza box that should have been transparent, revealing the underlying surface, were rendered as opaque black areas. I tried several solutions, including cropping out the black areas, adjusting the object's base colour, and manually editing the texture files before re-importing them. However, none of these attempts worked. Eventually, I had to remove the problematic textures and replace them with solid colours. I suspect that the cause may be related to the alpha channel, shader, or material settings, although I have not been able to identify the exact problem. I also noticed that my teammates encountered similar issues. In the future, I hope we can work together to investigate the cause of these transparency problems.



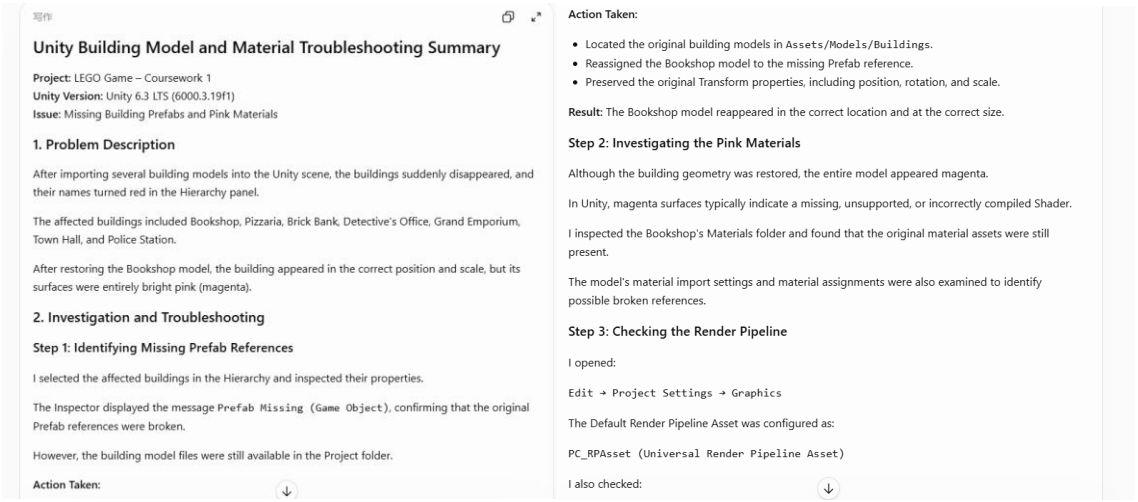




Another persistent problem involved building assets appearing pink in Unity, suggesting that their original materials were not rendering correctly. Following a solution I found online, I selected the materials and used the "Convert" option under the "Rendering" menu. This worked, and the original materials and colours reappeared. However, I then had to manually restore transparent materials, such as glass, because the conversion process had made them opaque. More frustratingly, when I switch between scenes and return, the buildings sometimes lose their materials again and turn pink. In some cases, they even disappear entirely, forcing me to repeat the repair process. This issue still occurs occasionally.

Following these stages of development, our game map now contains buildings and character models. Throughout the development process, I attended every meeting and tried to maintain effective communication with my teammates to improve the map. Moving forward, I intend to work more closely with the team members responsible for player movement and other systems, exchanging ideas and coordinating elements such as character scale, road layout, building placement, and future interaction requirements.

Regarding AI, I have mainly used it to translate and organise English-language tutorial content into step-by-step learning guides in Chinese. I have also used it for troubleshooting when I encounter problems while working in Unity or when my test results do not match my expectations. I do not want AI to generate an entire game project for me, leaving me with code that I do not understand, as this would provide little value to my learning. In practice, I use AI to translate English tutorials and turn video subtitles into clear, step-by-step instructions. Since I sometimes struggle to understand English tutorials or C# syntax, AI has been very helpful in organising my Chinese study notes and explaining difficult concepts.



写作

Unity Building Model and Material Troubleshooting Summary

Project: LEGO Game – Coursework 1
Unity Version: Unity 6.3 LTS (6000.3.19f1)
Issue: Missing Building Prefabs and Pink Materials

1. Problem Description

After importing several building models into the Unity scene, the buildings suddenly disappeared, and their names turned red in the Hierarchy panel.

The affected buildings included Bookshop, Pizzaria, Brick Bank, Detective's Office, Grand Emporium, Town Hall, and Police Station.

After restoring the Bookshop model, the building appeared in the correct position and scale, but its surfaces were entirely bright pink (magenta).

2. Investigation and Troubleshooting

Step 1: Identifying Missing Prefab References

I selected the affected buildings in the Hierarchy and inspected their properties.

The Inspector displayed the message `Prefab Missing (Game Object)`, confirming that the original Prefab references were broken.

However, the building model files were still available in the Project folder.

Action Taken:

Action Taken:

- Located the original building models in `Assets/Models/Buildings`.
- Reassigned the Bookshop model to the missing Prefab reference.
- Preserved the original Transform properties, including position, rotation, and scale.

Result: The Bookshop model reappeared in the correct location and at the correct size.

Step 2: Investigating the Pink Materials

Although the building geometry was restored, the entire model appeared magenta.

In Unity, magenta surfaces typically indicate a missing, unsupported, or incorrectly compiled Shader.

I inspected the Bookshop's Materials folder and found that the original material assets were still present.

The model's material import settings and material assignments were also examined to identify possible broken references.

Step 3: Checking the Render Pipeline

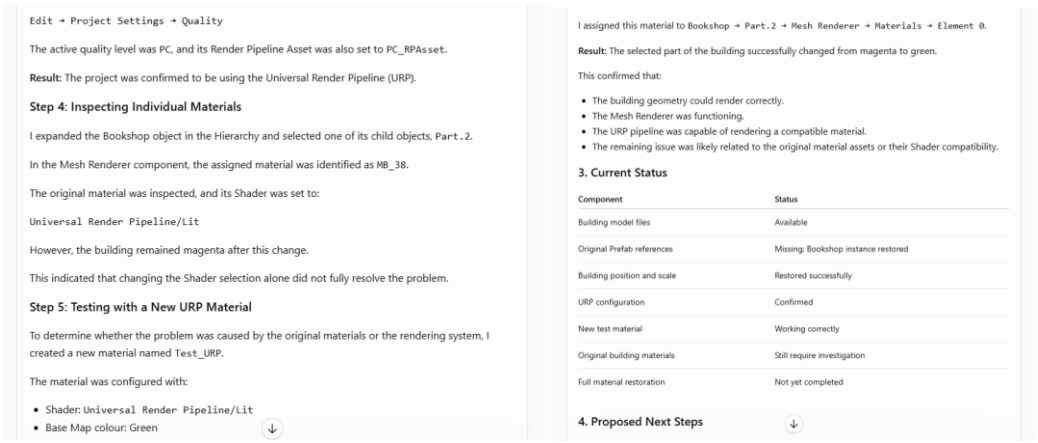
I opened:

`Edit → Project Settings → Graphics`

The Default Render Pipeline Asset was configured as:

`PC_RPAsset (Universal Render Pipeline Asset)`

I also checked:



`Edit → Project Settings → Quality`

The active quality level was PC, and its Render Pipeline Asset was also set to `PC_RPAsset`.

Result: The project was confirmed to be using the Universal Render Pipeline (URP).

Step 4: Inspecting Individual Materials

I expanded the Bookshop object in the Hierarchy and selected one of its child objects, `Part.2`.

In the Mesh Renderer component, the assigned material was identified as `MB_38`.

The original material was inspected, and its Shader was set to:

`Universal Render Pipeline/Lit`

However, the building remained magenta after this change.

This indicated that changing the Shader selection alone did not fully resolve the problem.

Step 5: Testing with a New URP Material

To determine whether the problem was caused by the original materials or the rendering system, I created a new material named `Test_URP`.

The material was configured with:

- Shader: `Universal Render Pipeline/Lit`
- Base Map colour: Green

I assigned this material to `Bookshop → Part.2 → Mesh Renderer → Materials → Element 0`.

Result: The selected part of the building successfully changed from magenta to green.

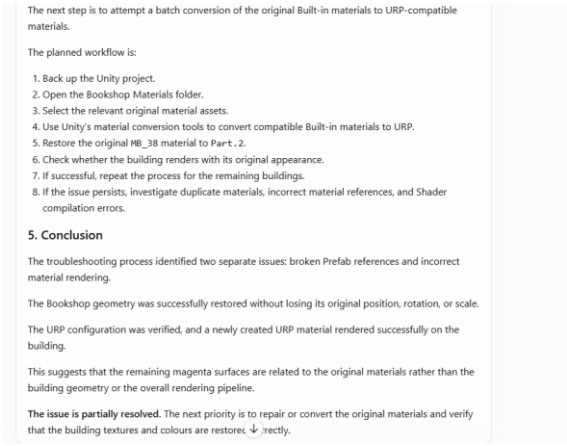
This confirmed that:

- The building geometry could render correctly.
- The Mesh Renderer was functioning.
- The URP pipeline was capable of rendering a compatible material.
- The remaining issue was likely related to the original material assets or their Shader compatibility.

3. Current Status

Component	Status
Building model files	Available
Original Prefab references	Missing; Bookshop instance restored
Building position and scale	Restored successfully
URP configuration	Confirmed
New test material	Working correctly
Original building materials	Still require investigation
Full material restoration	Not yet completed

4. Proposed Next Steps



The next step is to attempt a batch conversion of the original Built-in materials to URP-compatible materials.

The planned workflow is:

1. Back up the Unity project.
2. Open the Bookshop Materials folder.
3. Select the relevant original material assets.
4. Use Unity's material conversion tools to convert compatible Built-in materials to URP.
5. Restore the original `MB_38` material to `Part.2`.
6. Check whether the building renders with its original appearance.
7. If successful, repeat the process for the remaining buildings.
8. If the issue persists, investigate duplicate materials, incorrect material references, and Shader compilation errors.

5. Conclusion

The troubleshooting process identified two separate issues: broken Prefab references and incorrect material rendering.

The Bookshop geometry was successfully restored without losing its original position, rotation, or scale.

The URP configuration was verified, and a newly created URP material rendered successfully on the building.

This suggests that the remaining magenta surfaces are related to the original materials rather than the building geometry or the overall rendering pipeline.

The issue is partially resolved. The next priority is to repair or convert the original materials and verify that the building textures and colours are restored correctly.

When I encounter a problem during development, whether it is something I cannot solve or something I do not fully understand, I ask AI for guidance. After receiving a possible solution, I apply the suggested changes myself in Unity or my C# code, run tests, and provide further feedback based on the results. I repeat this process until the problem is resolved and I understand the underlying principles. I do not believe that AI's responses should be accepted without question. Even when using AI as a learning tool, I still need to understand and implement the code myself, and verify the compilation and runtime results. Furthermore, AI may sometimes suggest methods that are incompatible with my version of Unity or unsuitable for my specific project, potentially making a simple problem more complicated. Ultimately, I see AI as a valuable tutor, assistant, or on-demand guide, but it cannot do the learning for us. We must verify everything ourselves, step by step. It is much like having a treasure map: unless you actually start digging, you will never know whether there is any treasure to be found.